BACKEND AND DEVOPS ASSIGNMENT REPORT

AI-Powered Transaction Processing Pipeline

FastAPI, PostgreSQL, Redis, Celery, Docker Compose, and LLM-compatible enrichment

Async

Uploads return immediately while Celery performs background processing.

Durable

Jobs, transactions, anomalies, and summaries persist in PostgreSQL.

LLM Ready

Deterministic fallback by default with Gemini and Ollama paths.

Executive Summary

This project implements an end-to-end backend pipeline that accepts financial transaction CSV uploads, processes them asynchronously, and exposes job status and final results through a REST API.

The system is designed for reviewer-friendly execution with Docker Compose. Running docker compose up --build starts the API, worker, PostgreSQL database, and Redis broker without requiring local Python setup or manual database creation for the default path.

Problem Statement

Financial transaction exports are often inconsistent: dates appear in mixed formats, amounts contain symbols or separators, categories can be missing, records may be duplicated, and suspicious transactions need to be highlighted for review. This project converts raw CSV input into normalized, queryable, and review-ready transaction intelligence.

System Architecture

Component	Responsibility
FastAPI API	Validates CSV uploads, creates jobs, queues tasks, and serves status/results.
PostgreSQL	Stores job metadata, cleaned transactions, anomaly details, and summary payloads.
Redis	Acts as the Celery broker and result backend.
Celery Worker	Runs cleaning, anomaly detection, category classification, and summary generation.
LLM Client	Provides deterministic fallback plus Gemini and Ollama integration paths.

Request Lifecycle

- Client uploads a CSV to POST /jobs/upload.
- The API validates file type and UTF-8 content, creates a pending job, and enqueues a Celery task.
- The worker marks the job as processing, cleans rows, detects anomalies, and classifies missing categories.
- The worker stores cleaned transactions and a summary in PostgreSQL.
- The client polls status or reads full output from GET /jobs/{job_id}/results.

API Surface

Endpoint	Purpose
GET /health	Confirms API availability.
POST /jobs/upload	Accepts a CSV and starts asynchronous processing.
GET /jobs	Lists submitted jobs, optionally filtered by status.
GET /jobs/{job_id}/status	Returns current state and summary once available.
GET /jobs/{job_id}/results	Returns cleaned transactions, anomalies, category spend, and LLM summary.

Data Processing Logic

Cleaning

Drops duplicates, parses three date formats, normalizes amounts, uppercases status/currency, and marks blank categories.

Anomaly Detection

Flags 3x account median amounts, USD use at domestic-only merchants, and notes mentioning suspicious activity.

LLM and Fallback Strategy

The project uses an LLM-compatible client for category classification and summary generation. Docker Compose sets LLM_PROVIDER=fallback, so the default run is deterministic and does not require network access or API keys. External providers retry up to three times with exponential backoff.

Database Design

Table	Purpose
jobs	Tracks lifecycle and metadata for each uploaded CSV.
transactions	Stores each cleaned transaction linked to its job, including anomaly and LLM fields.
job_summaries	Stores totals, top merchants, risk level, category spend, and raw summary JSON.

Deployment and Operations

The full stack is defined in docker-compose.yml. The API and worker share the application image, while PostgreSQL and Redis use official Alpine images. PostgreSQL includes a health check so dependent services wait for the database before starting.

Primary run command: docker compose up --build

Clean reset: docker compose down -v, then docker compose up --build

Testing and Quality

The repository includes a focused processing test that validates duplicate removal, date parsing, currency/status normalization, fallback category classification, and anomaly detection.

Strengths

- Clear separation between API request handling and background processing.
- Dockerized reviewer experience with minimal setup friction.
- Durable job and result storage instead of in-memory state.
- Deterministic fallback behavior for reliable local demonstrations.
- Human-readable anomaly explanations preserved in the output model.

Future Enhancements

- Add Alembic migrations for production-grade schema evolution.
- Add authentication and per-user job isolation.
- Improve anomaly scoring with merchant, account, and time-window baselines.
- Add API-level integration tests with a test database and worker execution path.
- Add observability through structured logs, metrics, and task tracing.

Conclusion

The AI-Powered Transaction Processing Pipeline demonstrates a practical backend architecture for asynchronous financial data processing. It combines CSV normalization, anomaly detection, LLM-compatible enrichment, persistent storage, and Docker-based deployment into a cohesive system that is easy to run and evaluate.